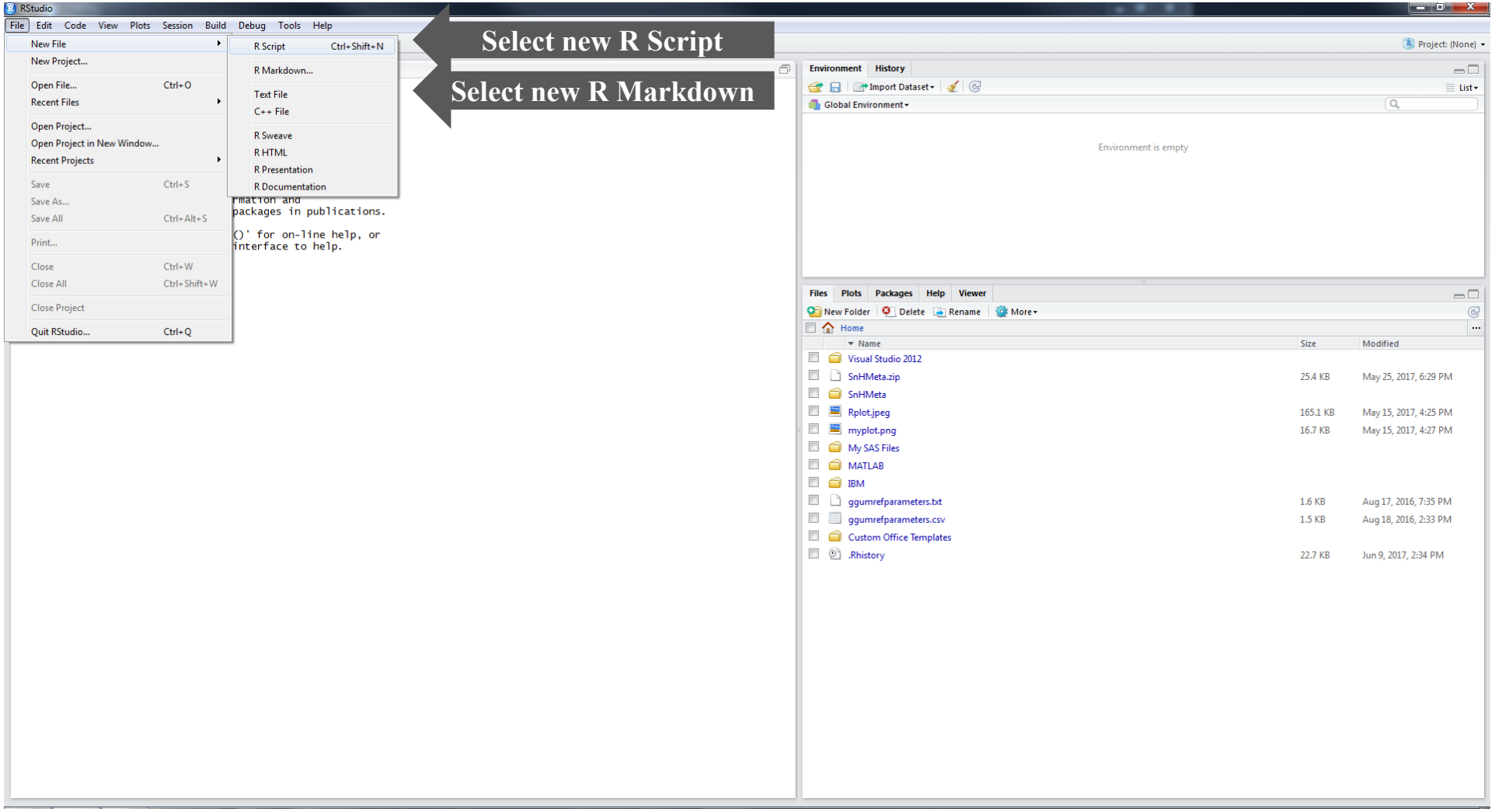
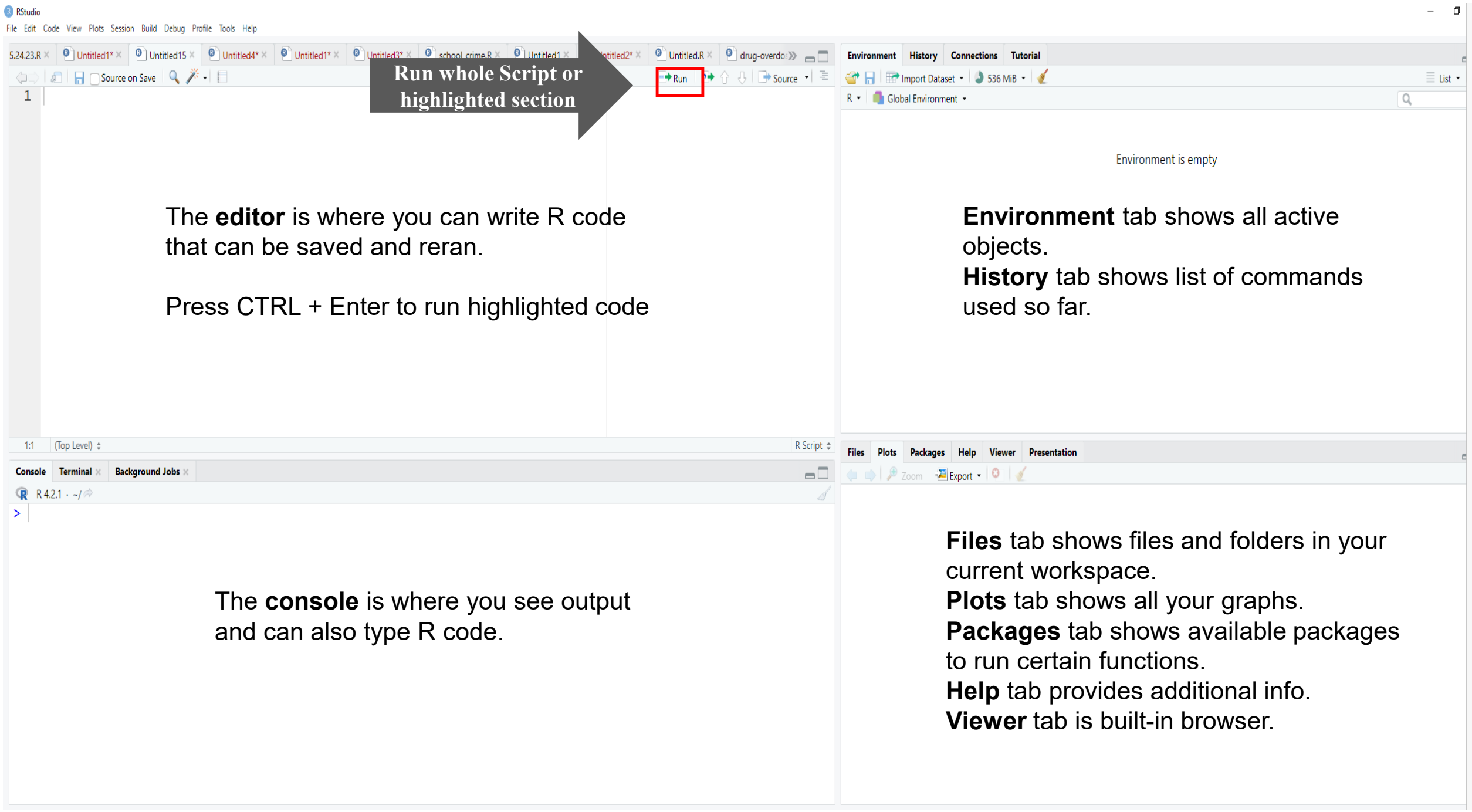
Using R for data wrangling

Reading and Cleaning Data

Exam 1

Starting Out





The **editor** is where you can write R code that can be saved and reran.

Press CTRL + Enter to run highlighted code

Environment tab shows all active objects.
History tab shows list of commands used so far.

The **console** is where you see output and can also type R code.

Files tab shows files and folders in your current workspace.
Plots tab shows all your graphs.
Packages tab shows available packages to run certain functions.
Help tab provides additional info.
Viewer tab is built-in browser.

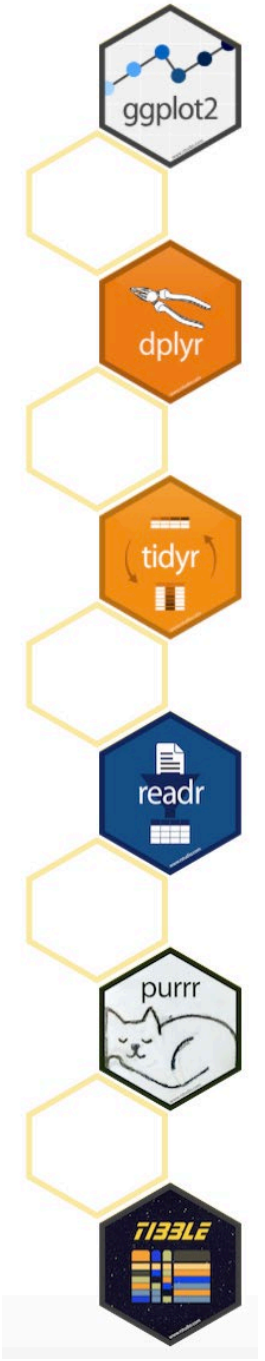
Getting started with packages

- Packages are extensions to base R. They contain additional functions, documentation for using them, and sample data.

- Packages are available from the Comprehensive R Archive Network (CRAN).
  - https://cran.r-project.org/web/packages/available_packages_by_name.html

- The “tidyverse” is a set of packages for data manipulation, exploration, and visualization. They share a common design and work in harmony (this also means that there is overlap which ***can*** cause issues).

```
#Install and load the package 'tidyverse'
install.packages(“tidyverse”) # only need to run once (the checkbox is a comment)
library(tidyverse) #run at start of every R session to use
```



https://www.tidyverse.org/

Important Libraries (not in Base R)

Data Wrangling	Plotting	Descriptive Statistics	Mapping/GIS
dplyr, tidyr, plyr	ggplot2	Hmisc	tmap & tmaptools
janitor	lattice	psych	ggmaps
tidyverse	patchwork	summarytools	sf
stringr		tableone	tidycensus
readr		desctable	
foreign		GGally	
remotes			

Note: R is **case sensitive**. The packages must be typed exactly as shown above.

Syntax for packages

- Developers of different packages may use the same function name.

- Good coding practice to specify package::function() to be explicit:

```
# Instead of:
filter()

# Specify the package for that function:
dplyr::filter()
```

Hint: after typing the package press the tab button to see a list of functions
Hint: after typing the object press the tab button to see a list of 'children'

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | evalsp23.classes.andrewheiss.com

ted.qmd | index.qmd | ps2_md.Rmd | moderation.qmd | day2.Rmd | day5.R

Render on Save | Render | Run

Source | Visual | Outline

```
1 ---
2 title: "Resources"
3 ---
4
5 On this page, you will find a number of
  resources to help you with the assignments.
6
```

5:84 (Top Level) Quarto

Console | Terminal | Jobs

R 4.3.2 · za-salerno.1/evalsp23.classes.andrewheiss.com/

```
> df$
```

- bcde11 [df]
- sexab [df]
- sex [df]
- tra1_mean [df]

<numeric> [1779]

num [1:1779] -11.42 20.58 -3.42 4.58 -17.42 ...

Press 'Tab'

Environment | History | Connections | Build | Git | Tutorial

Import Dataset | 554 MiB | List

R | Global Environment

Data	
all_risk	List of 4
contcont…	2 obs. of 9 variables
df	1779 obs. of 4 variabl…
earliest…	12 obs. of 5 variables
fit	List of 14
ggplot_t…	List of 9
interact	6 obs. of 6 variables
labels	List of 3
moderati…	2064 obs. of 5 variabl…
mylist	List of 2
p	List of 9
p1	List of 9
Plot_Int…	List of 9
timeline…	12 obs. of 5 variables
total	num [1:2, 1:2] 47 168 …
Values	
category…	"Political.Party"
label_co…	"Prime.Minister"
party_co…	chr [1:4] "#DC241f" "#0…
Functions	
gantt_la…	function (start_date =…

Files | Plots | Packages | Help | Viewer | Presentation

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | evalsp23.classes.andrewheiss.c

ted.qmd | index.qmd | ps2_md.Rmd | moderation.qmd | day2.Rmd | day5.R

Render on Save | Render | Run

Source | Visual | Outline

```
1 ---
2 title: "Resources"
3 ---
4
5 On this page, you will find a number of
  resources to help you with the assignments.
6
```

5:84 (Top Level) Quarto

Console | Terminal | Background

R 4.3.2 · C:/Users/barbo… 23.classes.andrewheiss.com/

```
> ggplot2::
```

- aes {ggplot2}
- aes_ {ggplot2}
- aes_all {ggplot2}
- aes_auto {ggplot2}
- aes_q {ggplot2}
- aes_string {ggplot2}
- after_scale {ggplot2}

aes_all(vars)

Given a character vector, create a set of identity mappings

Given a character vector, create a set of identity mappings

Press F1 for additional help

Press 'Tab'

Environment | History | Connections | Build | Git | Tutorial

Import Dataset | 560 MiB | List

R | Global Environment

Data	
all_risk	List of 4
contcont…	2 obs. of 9 variables
df	1779 obs. of 4 variabl…
earliest…	12 obs. of 5 variables
fit	List of 14
ggplot_t…	List of 9
interact	6 obs. of 6 variables
labels	List of 3
moderati…	2064 obs. of 5 variabl…
mylist	List of 2
p	List of 9
p1	List of 9
Plot_Int…	List of 9
timeline…	12 obs. of 5 variables
total	num [1:2, 1:2] 47 168 …
Values	
category…	"Political.Party"
label_co…	"Prime.Minister"
party_co…	chr [1:4] "#DC241f" "#0…
Functions	
gantt_la…	function (start_date =…

Files | Plots | Packages | Help | Viewer | Presentation

Data structure

- Two helpful slogans to understand R computations
  - Anything that *exists* is an object
    - Dataset
    - The results of a statistical test
    - Graph
  - Anything that happens is a function call
- R has several types of data values
  - You'll likely deal with these the three in the table

Type	Example
Character	"A", "b", "Portland, OR"
Numeric	2, 15.5
Logical	TRUE, FALSE

Data Structures

- Vector
  - Values of all same data type

Numeric

1
2
3

Character

M
F
M

```
Console ~/
>
> #Numeric Vector
> x <- c(1,2,3)
> x
[1] 1 2 3
> class(x)
[1] "numeric"
>
> #Character Vector
> y <- c("F","M","F")
> y
[1] "F" "M" "F"
> class(y)
[1] "character"
>
```

Data Structures

- Factor
  - Collection of values that have fixed set of possible values
  - Similar to vector
  - Represents *categorical* variables

M
F
M

```
Console ~/
> #Factor
> y <- factor(c("F","M","F"))
> y
[1] F M F
Levels: F M
> class(y)
[1] "factor"
>
```

Your character variables must be factored in some instances, or you will get an error!!

```
> y <- c("F", "M", "F")
> y <- as.data.frame(y)
> y
  y
1 F
2 M
3 F
> colnames(y)[1]
[1] "y"
> colnames(y)[1] <- "Sex"
> colnames(y)[1] <- "Sex_at_Birth"
> colnames(y)[1] <- "Sex at Birth"
> y$`Sex at Birth` <- as.factor(y$`Sex at Birth`)
> levels(y$`Sex at Birth`)
[1] "F" "M"
> levels(y$`Sex at Birth`) <- c("Female", "Male")
> levels(y$`Sex at Birth`)
[1] "Female" "Male"
```

Data Structures

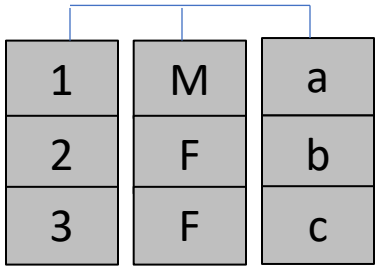
- Matrices
  - Two-dimensional collection of values of same type
    - Array can extend this to more than two dimensions

1	4	2
7	4	4
9	2	3

```
Console ~/
> x <- matrix(c(1,7,9,4,4,2,2,4,3), nrow=3, ncol=3)
> x
     [,1] [,2] [,3]
[1,]    1    4    2
[2,]    7    4    4
[3,]    9    2    3
> class(x)
[1] "matrix"
>
```

Data Structures

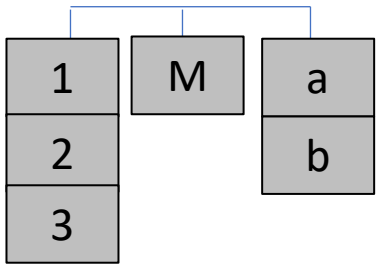
- Data frames
  - Collection of vectors that *all have same length*
  - Columns can have different data types
    - Array can extend this to more than two dimensions




```
Console ~/
> #Data Frame
> x <- c(1,2,3)
> y <- factor(c("M","F","F"))
> z <- c("a","b","c")
> xyz <- data.frame(x,y,z)
> xyz
  x y z
1 1 M a
2 2 F b
3 3 F c
> class(xyz)
[1] "data.frame"
>
```

Data Structures

- Lists
  - Similar to data frame, but columns can have varying lengths

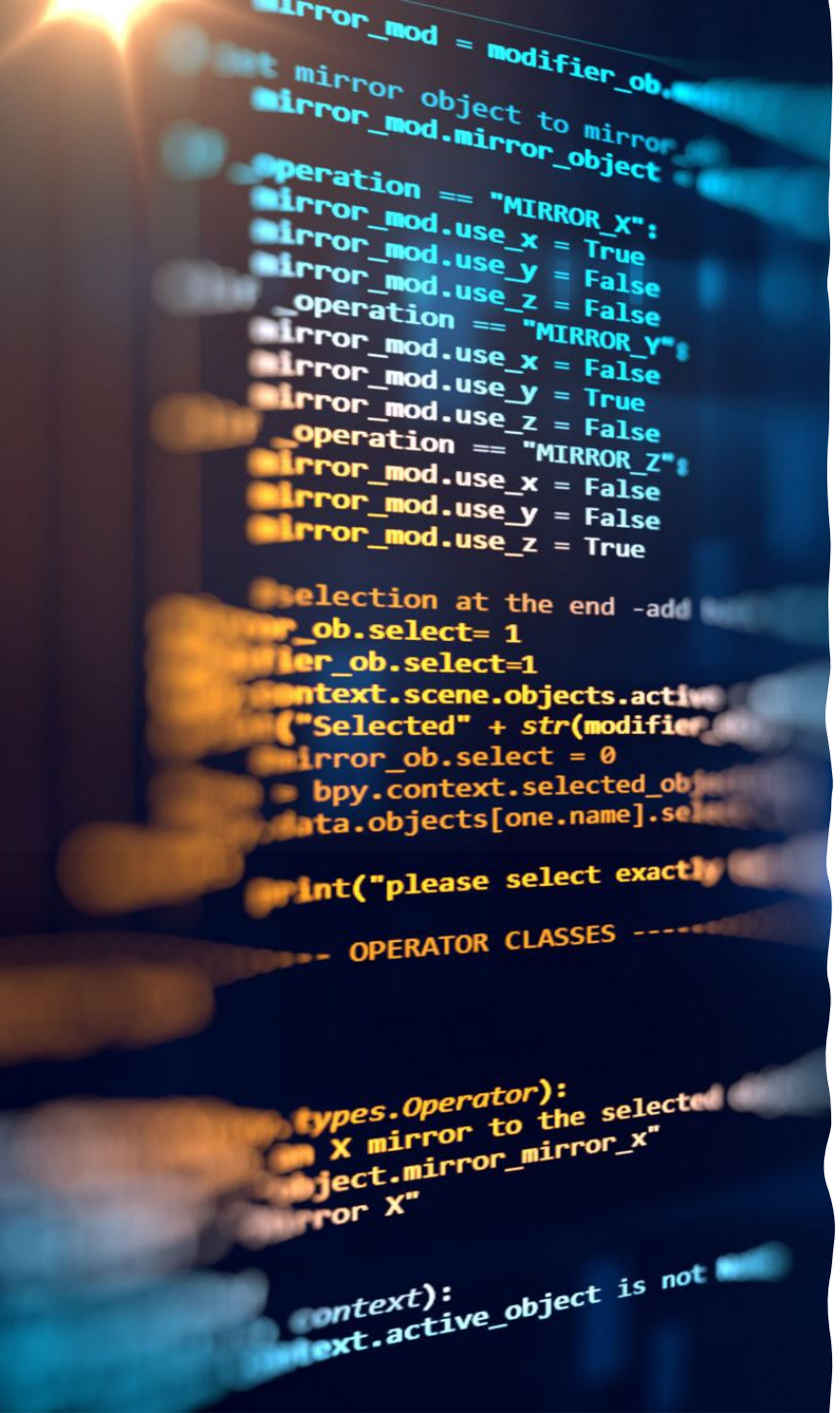



```
Console ~/
> #List
> x <- c(1,2,3)
> y <- factor(c("M"))
> z <- c("a","b")
> xyz <- list(x,y,z)
> xyz
[[1]]
[1] 1 2 3

[[2]]
[1] M
Levels: M

[[3]]
[1] "a" "b"

> class(xyz)
[1] "list"
>
```



Things to Note When Writing Code

- Syntax is *case*-sensitive
  - Typing in “Mydf” to call an object named “mydf” is wrong
  - Object names consist of letters, numbers, underscores “_” and periods “.”
- You can use either ‘<-’ or ‘=‘ to assign objects value
  - a <- c(1,2,3)
  - a = c(1,2,3) ← do not do
- Functions can stand on their own or be an object
  - 1 + 2
  - a <- 1 + 2
  - Important to consider later on when we do analyses

Create a simple dataset in R (by hand)

```
sex_at_birth <- c("male", "male", "female", "male", "female", "female")
age <- c(10, 12, 3, 0, 1, 16)
cbl_score <- c(50, 39, 48, 54, 70, 55)
dep_score <- c(40, 29, 37, 44, 60, 45)
total_score <- cbl_score + dep_score
mydat <- data.frame(sex_at_birth, age, cbl_score, dep_score, total_score)
```

```
str(mydat)
View(mydat)
```

Use the 'str' command to check what the computer understands to be the contents of your data-this matters when you analyze ('factor' plots and adds differently than 'integer' or 'number')

Things to note

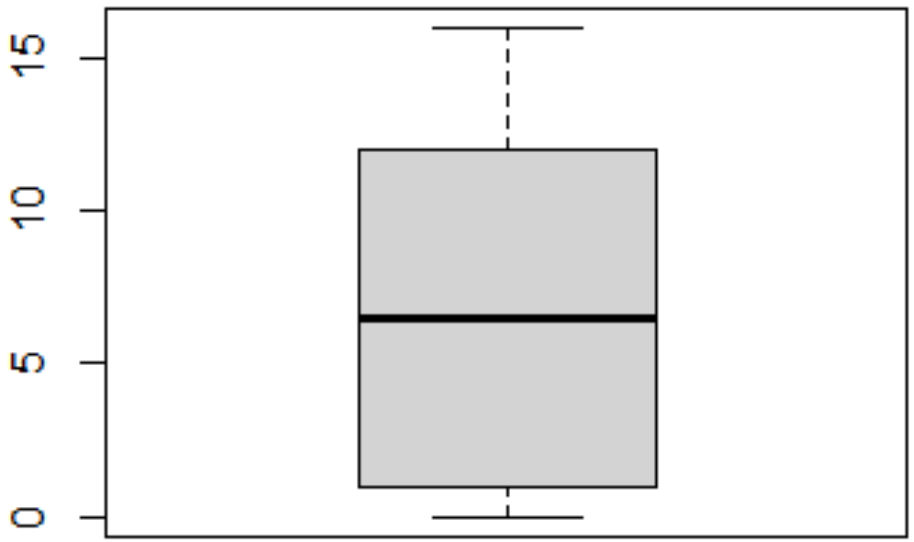
WARNING: DO NOT COPY AND PASTE

```
table(mydat$sex_at_birth)
```

Basic Plotting (Base R)

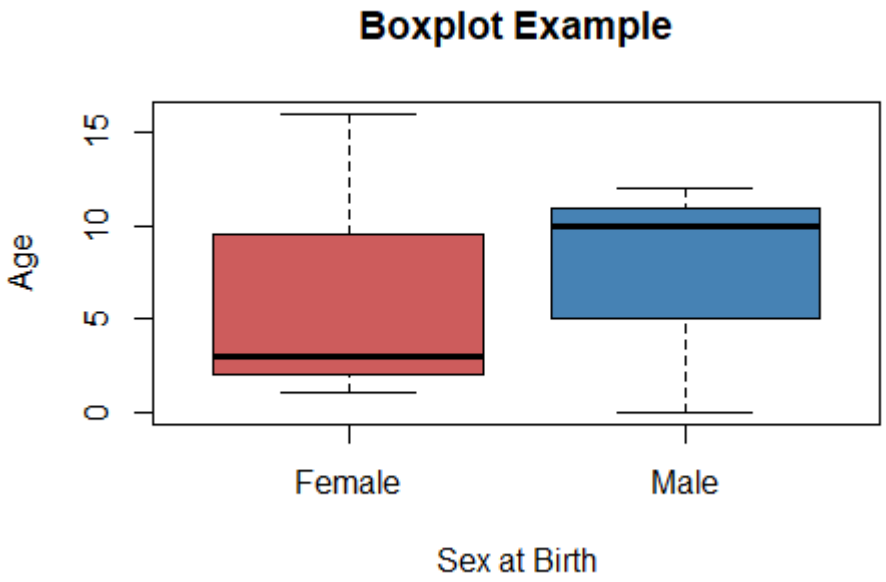
The plot and hist functions present users with options to customize their graphs (note: R has extensive help pages or search web for examples)

```
plot(mydat$sex_at_birth)
```



Default: Awful!

```
boxplot(age ~ sex_at_birth,
  data=mydat,
  ylab="Age",
  xlab = "Sex at Birth",
  col=c("red", "steelblue"),
  names = c("Female", "Male"),
  main = "Boxplot Example")
```

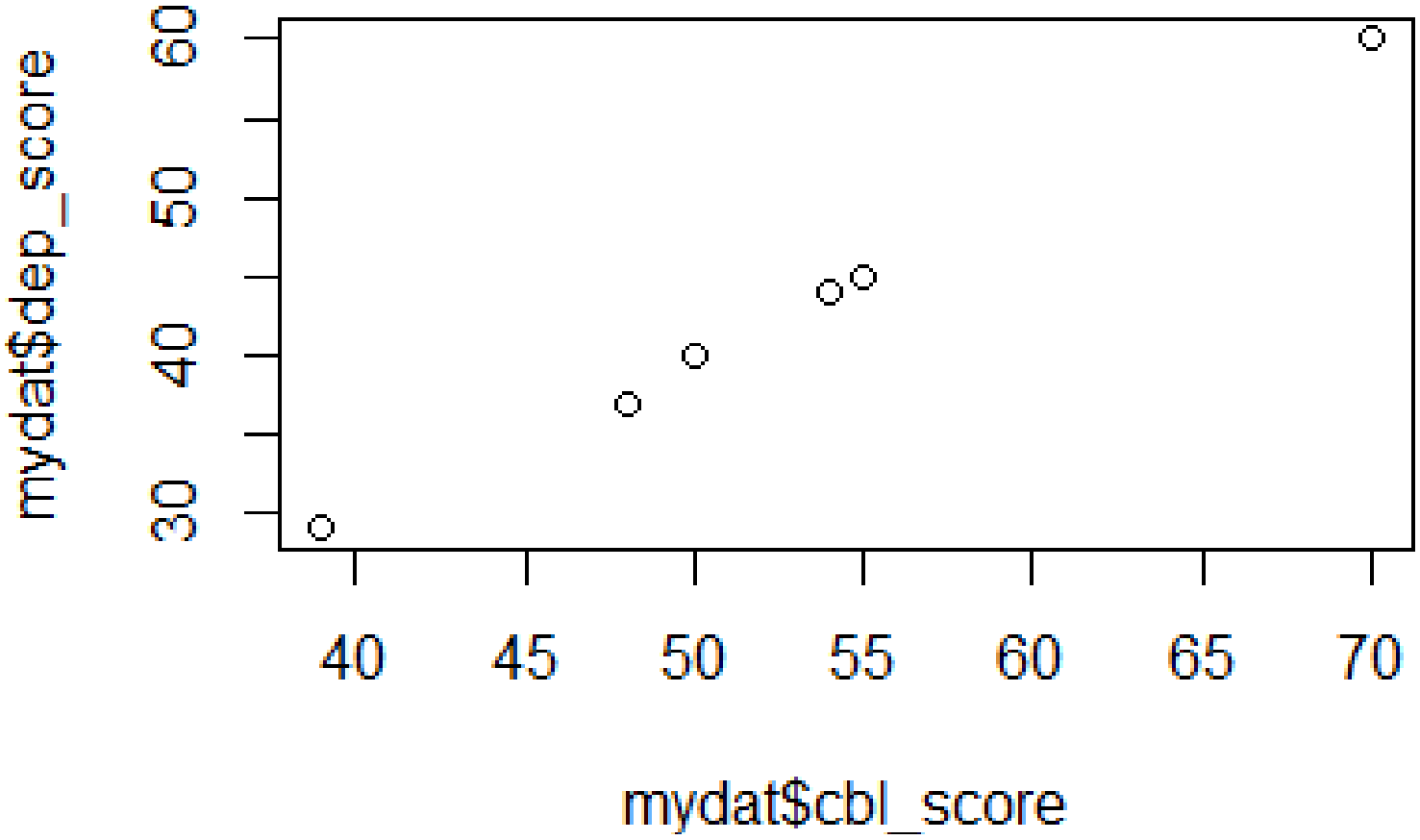


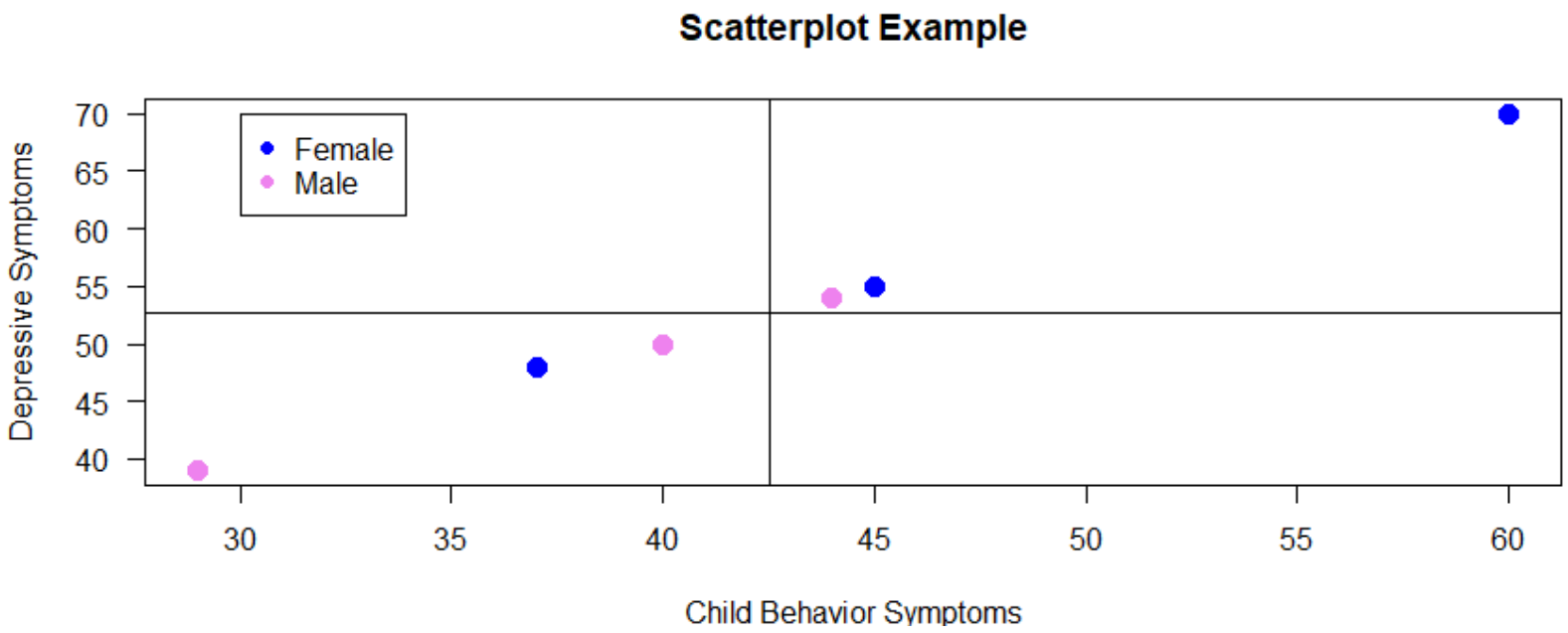
Nice!

Take it one step at a time

```
boxplot(mydat$age, xlab = "Age", col = "steelblue")
boxplot(mydat$age ~ mydat$sex_at_birth,  ylab="Age")
boxplot(age ~ sex_at_birth, data=mydat, ylab="Age")
boxplot(age ~ sex_at_birth, data=mydat, ylab="Age", col="steelblue")
boxplot(age ~ sex_at_birth, data=mydat, ylab="Age", col=c("indianred", "steelblue")
```

Scatterplots






```
plot(cbl_score ~ dep_score,
  data=mydat,
  pch=19,
  col = c("blue", "violet")[mydat$sex_at_birth],
  ylab="Depressive Symptoms",
  xlab="Child Behavior Symptoms",
  las=1,
  cex = 1.5,
  main = "Scatterplot Example")

legend(30,70,c("Female","Male"),col=c("blue", "violet"),pch=19)

abline(h=mean(mydat$cbl_score),v=mean(mydat$dep_score))
```

Importing Data

- R can read a variety of datasets
- The most important dataset is a csv file

mydat <- read.csv(“PATH TO FILE”)

mydat <- read.csv(“C:/Users/barboza-salerno.1/Documents/mydat.csv”)

NOTE: R is case sensitive and uses forward slashes

Note: the file NAME must be in quotes

Also, once you type the “/” you can hit the tab key to autofill

```
mydat <- read.csv("mydat.csv")
mydat <- read.csv("C:/Users/barboza-salerno.1/OneDrive - The Ohio
State University/Documents/mydat.csv”)
```

dfs and tibbles

```r
setwd("C:/Users/barboza-salerno.1/Downloads/")
moderation_example = read.spss("moderation.sav", use.value.labels = F, to.data.frame = T)
str(moderation_example)
```

```
'data.frame':   2064 obs. of  5 variables:
 $ NSCAWID: num  2e+05 2e+05 2e+05 2e+05 2e+05 ...
 $ tra1   : num  39 36 71 47 55 33 61 64 51 58 ...
 $ cidi1  : num  50 49 74 46 39 46 63 54 53 60 ...
 $ bcdel1 : num  56 73 50 61 50 50 70 51 57 61 ...
 $ sexab  : num  0 NA 1 0 0 0 0 NA 0 1 ...
 - attr(*, "variable.labels")= Named chr [1:5] "NSCAW Identifier" "TR: Trauma: PTS T Score" "CD: Depression: Total CDI Std. Score" "BC: Delinquent Behavior Std" ...
  ..- attr(*, "names")= chr [1:5] "NSCAWID" "tra1" "cidi1" "bcdel1" ...
 - attr(*, "codepage")= int 65001
```

```r
setwd("C:/Users/barboza-salerno.1/Downloads/")
moderation_example = tibble::as_tibble(read.spss("moderation.sav", use.value.labels = F, to.data.frame = T))
str(moderation_example)
```

```
tibble [2,064 × 5] (S3: tbl_df/tbl/data.frame)
 $ NSCAWID: num [1:2064] 2e+05 2e+05 2e+05 2e+05 2e+05 ...
 $ tra1   : num [1:2064] 39 36 71 47 55 33 61 64 51 58 ...
 $ cidi1  : num [1:2064] 50 49 74 46 39 46 63 54 53 60 ...
 $ bcdel1 : num [1:2064] 56 73 50 61 50 50 70 51 57 61 ...
 $ sexab  : num [1:2064] 0 NA 1 0 0 0 0 NA 0 1 ...
 - attr(*, "variable.labels")= Named chr [1:5] "NSCAW Identifier" "TR: Trauma: PTS T Score" "CD: Depression: Total CDI Std. Score" "BC: Delinquent Behavior Std" ...
  ..- attr(*, "names")= chr [1:5] "NSCAWID" "tra1" "cidi1" "bcdel1" ...
 - attr(*, "codepage")= int 65001
```

Importing Data Using Syntax

```r
#Helpful syntax for clearing out R workspace
rm(list=ls()) #Cleans (empties) global environment
dev.off()     #Cleans plots/graphics
cat("\014")   #Cleans out the console


#Find the file you want to read
data.file <- file.choose()
#Change the working directory to match file so you don't have to specify path to file
setwd(dirname(data.file))

#need to specify correct read function
#read a csv file
data <- read.csv(data.file, header=TRUE)

#read a txt file
data <- read.table(data.file, header=TRUE)
data

#read an xlsx file, need xlsx package installed and loaded
data <- read.xlsx(data.file, sheetName="Sheet1", header=FALSE)
data
```

Example .csv import

```
> # find the file you want to read
> data.file <- file.choose()
> # change the working direct to match file so you don't have to specify path to file
> setwd(dirname(data.file))
> #need to specify correct read function
> #read a csv file
> data <- read.csv(data.file, header=FALSE)
> data
  V1 V2 V3
1  1  M  a
2  3  F  b
3  3  M  c
>
```

Other file types

- Excel

```
library(xlsx)
mydat <- read.xlsx("name of file.xlsx", sheetName = "Sheet 1", header = T)
```

- SPSS, STATA, SAS

```
library(haven)
mydat <- read_sav("PATH_OR_URL_TO_SAS_FILE/file.sav")
mydat <- read_sas("
'PATH_OR_URL_TO_SAS_FILE/file.sas7bdat")
mydat <- read_dta('PATH_OR_URL_TO_STATA_FILE
/file.dta')
```

2. Intro to the tidyverse

The Tidyverse

- A collection of packages for data manipulation, exploration, and visualization.

- Share a common philosophy of R programming and work in harmony.

- Core tidyverse packages:

  readr dplyr tidyr ggplot2 tibble purrr

Data Manipulation with **dplyr**

- Very useful package for exploring and managing your data.

- *"A grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges."*

- Resources for getting started:
  - http://dplyr.tidyverse.org/
  - https://cran.r-project.org/web/packages/dplyr/vignettes/dplyr.html
  - https://www.rstudio.com/wp-content/uploads/2015/02/data-wrangling-cheatsheet.pdf
  - http://r4ds.had.co.nz/transform.html

dplyr part of the tidyverse

3. dplyr code structure

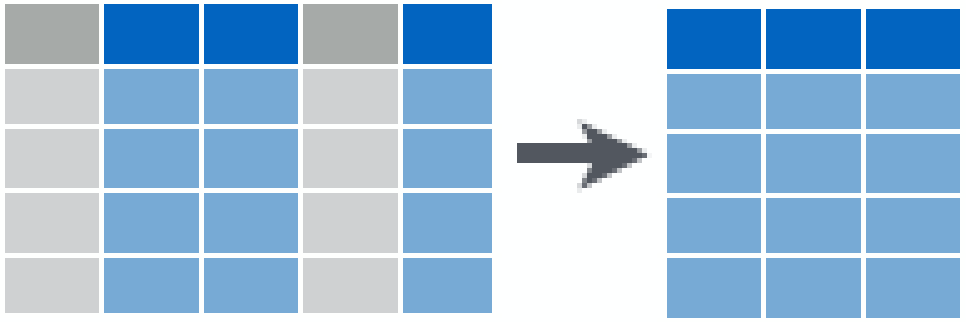
dplyr Key Functions

- select()
- filter()
- arrange()
- summarise()
- mutate()
- group_by()

More helpful info here: https://www.rstudio.com/wp-content/uploads/2015/02/data-wrangling-cheatsheet.pdf

dplyr Key Functions

- select()
  - Picks variables (columns) based on their names.



```
df %>% dplyr::select(c(2:3, 5))
```

```
names(df)
names(df) [1] "bcdel1" "sexab" "sex" "tra1_mean"
```

df %>% dplyr::select("sex")

df %>% dplyr::select("sex", "tra1_mean")

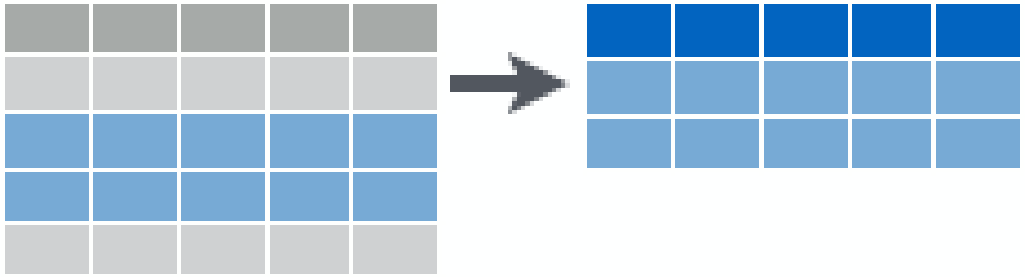
```
df_sub <- df %>% dplyr::select("sex", "tra1_mean")
names(df_sub) [1] "sex" "tra1_mean"
```

Query: why will the following code throw an error?

```
df %>% dplyr::select(c(2:3, 5))
```

dplyr Key Functions

- filter()
  - Picks observations (rows) based on their values.



```
df %>% filter(tra1_mean < 4)
```

dplyr Key Functions

- arrange()
  - Changes the ordering of the rows based on their values.

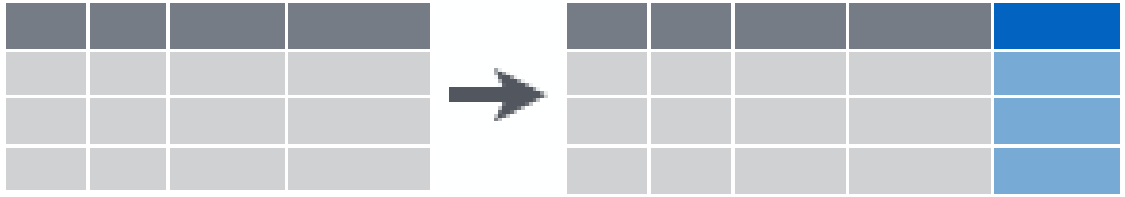
dplyr Key Functions

- summarise()
  - Reduces multiple values down to a single summary value.

```
df %>% summarise(del = mean(bcdel1, rm.na=T)) # A tibble: 1 × 1 del <dbl> 1 NA
df %>% summarise(del = mean(bcdel1, na.rm =T)) # A tibble: 1 × 1 del <dbl> 1 60.8
```

dplyr Key Functions

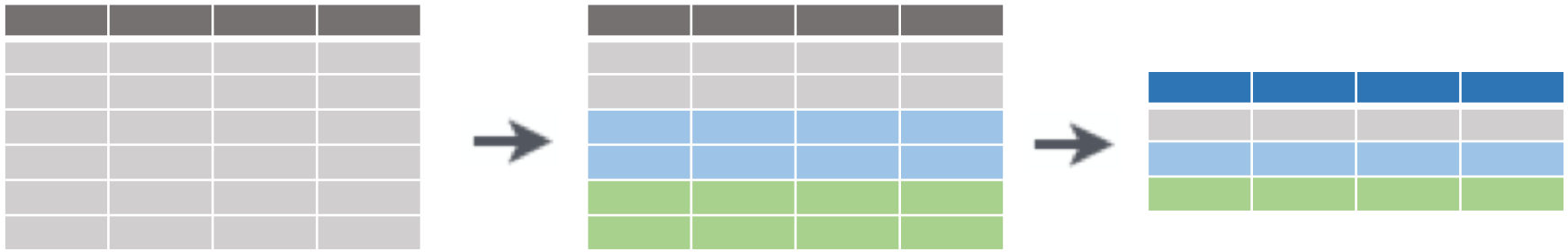
- mutate()
  - Adds new variables that are functions of existing variables.



```
df %>% mutate(del = bcdel1 - mean(bcdel1, na.rm =T))
```

dplyr Key Functions

- group_by()
  - Performs data operations on groups that are defined by variables.



```
> df %>% group_by(sex) %>% summarise(mean(bcdel1, na.rm =T))
# A tibble: 2 × 2
  sex   `mean(bcdel1, na.rm = T)`
  <chr>                     <dbl>
1 No                         60.5
2 Yes                        62.2
```

Key Functions

select()

Picks variables (columns) based on their names

filter()

Picks observations (rows) based on their values

arrange()

Changes the ordering of the rows

summarise()

Reduces multiple values down to a single summary value

mutate()

Adds new variables that are functions of existing variables

group_by()

Performs data operations on groups that are defined by variables

Key Operator: The Pipe %>%

- A common paradigm in all tidyverse R libraries is to use the pipe operator, %>%, which allows us to chain or pipe functions together
- Enables you to pass the object on left hand side as first argument of function on the right-hand side
- Goal of making our code more efficient and easier to read

Basic Structure

- Use the key functions and pipe to chain together multiple simple steps to achieve a more complicated result.

```
Dataset %>%
        Select rows or filter columns %>%
        Arrange or group the data %>%
        Calculate statistics or new variables of interest
```

Basic Structure

```
#Prints output to the console:
Dataset %>%
        Select rows or columns %>%
        Arrange or group the data %>%
        Calculate statistics or new variables
#Creates a new R object:
new_obj <-      Dataset %>%
                        Select rows or columns %>%
                        Arrange or group the data %>%
                        Calculate statistics or new variables
```

Reading files from the web

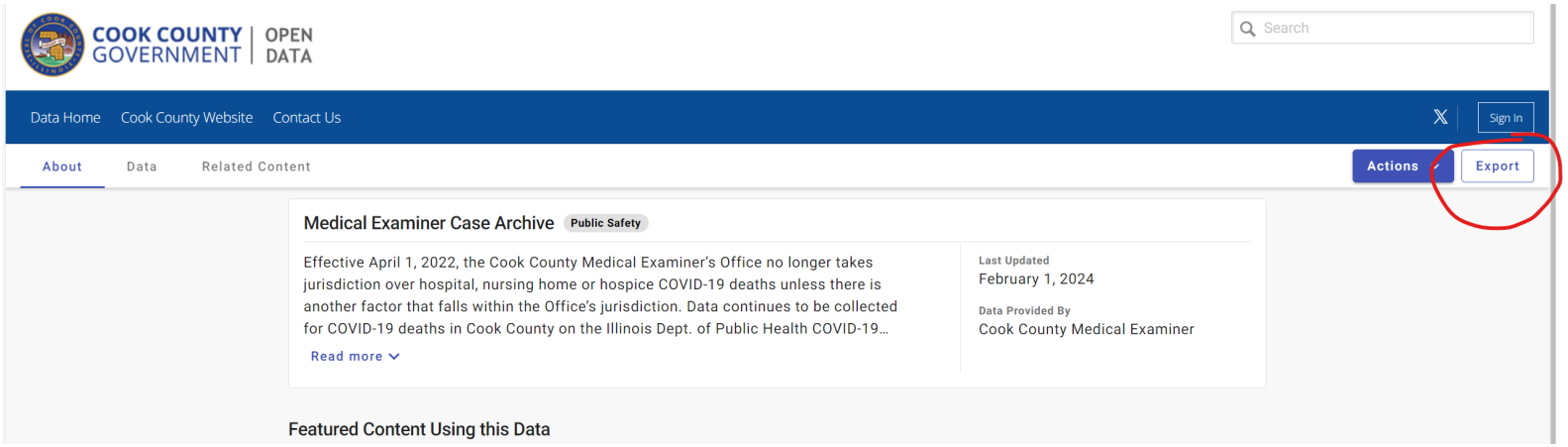
You are interested in exploring the relationship between early race and infant mortality in the city of Chicago. You need to find data. You remember that you had this amazing professor for research methods who told you about the Cook County medical examiner's case file. You decide to impress your colleagues at the Ivy league school where you were recently tenured (and secured the schools largest NIH award!). First, begin by downloading, preparing and exploring some key variables by race.

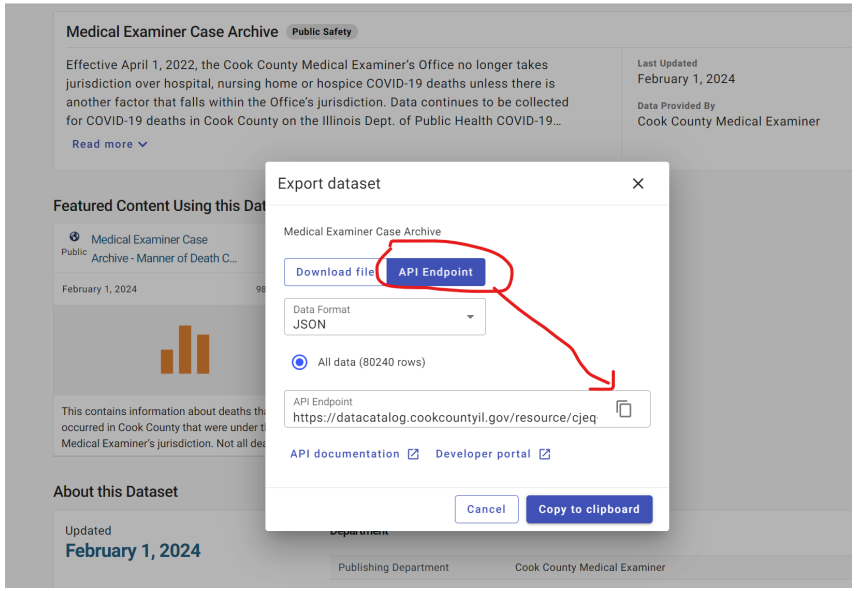
- **Data manipulation:** Create an analytic dataset that is a subset of the observations and variables in the original death data, specific to your research question.

- **Summary statistics:** Explore the variables corresponding to race, and manner of death.

Reading files from the web

Medical Examiner Case Archive | Cook County Open Data (cookcountyil.gov)





https://datacatalog.cookcountyil.gov/resource/cjeq-bs86.json

```
library(RSocrata)
options(scipen=10000)
#https://datacatalog.cookcountyil.gov/api/odata/v4/cjeq-bs86
child_homicide <-
read.socrata("https://datacatalog.cookcountyil.gov/api/odata/v4/cjeq-bs86")
```